

Algorithmie – Algorithmes de Tri

Table des matières

1 - Pourquoi trier ?	2
Faciliter la recherche	2
Organiser les données	2
Exemples du quotidien	2
2 - Tri à bulles (Bubble Sort)	4
Principe : comparer et échanger des éléments adjacents	4
Visualisation étape par étape	4
Notion de "passage" (tour complet)	6
Algorithme complet avec boucles imbriquées	6
Optimisation : arrêt si déjà trié	7
3 - Tri par sélection (Selection Sort)	8
Principe : trouver le minimum et le placer	8
Différence avec tri à bulles	8
Algorithme complet	9
Visualisation détaillée	10
4 - Comparaison des deux méthodes	13
Nombre de comparaisons	13
Lisibilité	13
Introduction intuitive à la complexité	14
5 - Exercices pratiques	15
Exercice 1 : Tri à bulles sur petit tableau	15
Exercice 2 : Tri par sélection guidé	15
Exercice 3 : Trier puis chercher	15

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

1- Pourquoi trier ?

Faciliter la recherche

Nous avons vu ce matin que la **recherche dichotomique** est très efficace.

Mais elle nécessite un tableau **trié**.

Donc trier un tableau permet d'accélérer les recherches futures.

Organiser les données

Le tri permet de :

- **Classer** des éléments par ordre logique
- **Hiérarchiser** : du plus petit au plus grand, du moins cher au plus cher
- **Prioriser** : les tâches urgentes en premier
- **Visualiser** : comprendre rapidement les données

Exemples du quotidien

1. Classer des notes

Non trié : [12, 8, 15, 10, 18, 9]

Trié : [8, 9, 10, 12, 15, 18]

→ On voit facilement qui a la meilleure/moins bonne note

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

2. Ranger des fichiers par date

Non trié : [2023-05-12, 2024-01-03, 2023-11-20]

Trié : [2023-05-12, 2023-11-20, 2024-01-03]

→ Chronologie claire

3. Trier des produits par prix

Non trié : [29€, 15€, 45€, 12€]

Trié : [12€, 15€, 29€, 45€]

→ Trouver rapidement le moins cher

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

2- Tri à bulles (Bubble Sort)

Principe : comparer et échanger des éléments adjacents

Le **tri à bulles** est un des algorithmes de tri les plus simples à comprendre.

Principe :

- On compare deux éléments consécutifs
- Si ils sont dans le mauvais ordre, on les échange
- On répète jusqu'à ce que le tableau soit trié

Pourquoi "à bulles" ?

Les grands éléments "remontent" vers la fin comme des bulles dans l'eau.

Visualisation étape par étape

Trions le tableau : `[5, 2, 8, 1, 9]`

Passage 1 :

```
[5, 2, 8, 1, 9]
  ↓           Comparer 5 et 2 → échanger
[2, 5, 8, 1, 9]
  ↓           Comparer 5 et 8 → OK
[2, 5, 8, 1, 9]
    ↓         Comparer 8 et 1 → échanger
[2, 5, 1, 8, 9]
      ↓       Comparer 8 et 9 → OK
[2, 5, 1, 8, 9]
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Après le passage 1, le **plus grand élément (9)** est à sa place finale.

Passage 2 :

```
[2, 5, 1, 8, 9]
  ↓          Comparer 2 et 5 → OK
[2, 5, 1, 8, 9]
    ↓          Comparer 5 et 1 → échanger
[2, 1, 5, 8, 9]
      ↓          Comparer 5 et 8 → OK
[2, 1, 5, 8, 9]
        ↓          Comparer 8 et 9 → OK (déjà en place)
[2, 1, 5, 8, 9]
```

Après le passage 2, les **deux plus grands éléments (8 et 9)** sont en place.

Passage 3 :

```
[2, 1, 5, 8, 9]
  ↓          Comparer 2 et 1 → échanger
[1, 2, 5, 8, 9]
    ↓          Comparer 2 et 5 → OK
[1, 2, 5, 8, 9]
      ↓          Comparer 5 et 8 → OK
[1, 2, 5, 8, 9]
```

Tableau trié !

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE

☒ Sophie POULAKOS

☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx

Algorithmie – Algorithmes de Tri

Notion de "passage" (tour complet)

Un **passage** est un parcours complet du tableau où on compare et échange les éléments adjacents.

À chaque passage :

- Au moins un élément arrive à sa position finale
- Le plus grand élément non trié "remonte" vers la fin

Nombre de passages nécessaires :

Pour un tableau de N éléments, on a besoin de **N-1 passages maximum**.

Algorithme complet avec boucles imbriquées

```
FONCTION triABulles(tableau)
  n ← longueur(tableau)

  POUR i de 0 à n - 2 // Nombre de passages
    POUR j de 0 à n - 2 - i // Parcours du tableau
      SI tableau[j] > tableau[j + 1]
        // Échanger les éléments
        temp ← tableau[j]
        tableau[j] ← tableau[j + 1]
        tableau[j + 1] ← temp
      FIN SI
    FIN POUR
  FIN POUR
FIN FONCTION
```

Explication :

- **Boucle externe (i)** : nombre de passages
- **Boucle interne (j)** : parcourt et compare les éléments adjacents
- **'n - 2 - i'** : on réduit la zone de comparaison car les derniers éléments sont déjà triés

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Optimisation : arrêt si déjà trié

On peut améliorer l'algorithme en arrêtant si aucun échange n'a été fait :

```
FONCTION triABullesOptimise(tableau)
  n ← longueur(tableau)

  POUR i de 0 à n - 2
    echange ← faux // Flag pour détecter les échanges

    POUR j de 0 à n - 2 - i
      SI tableau[j] > tableau[j + 1]
        // Échanger
        temp ← tableau[j]
        tableau[j] ← tableau[j + 1]
        tableau[j + 1] ← temp
        echange ← vrai
      FIN SI
    FIN POUR

    SI echange = faux
      // Aucun échange → tableau déjà trié
      SORTIR de la boucle
    FIN SI
  FIN POUR
FIN FONCTION
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

3- Tri par sélection (Selection Sort)

Principe : trouver le minimum et le placer

Le **tri par sélection** fonctionne différemment :

1. On cherche le **plus petit** élément du tableau
2. On le place au **début**
3. On cherche le plus petit élément **restant**
4. On le place en **deuxième position**
5. On répète jusqu'à avoir tout trié

Analogie : trier des cartes en cherchant toujours la plus petite restante.

Différence avec tri à bulles

Tri à Bulles	Tri par Sélection
Compare les éléments adjacents	Cherche le minimum dans tout le reste
Beaucoup d'échanges	Peu d'échanges (1 par passage)
Les grands éléments remontent	Les petits éléments sont placés

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Algorithme complet

```
FONCTION triParSelection(tableau)
    n ← longueur(tableau)

    POUR i de 0 à n - 2
        // Trouver l'index du minimum dans le reste du tableau
        indexMin ← i

        POUR j de i + 1 à n - 1
            SI tableau[j] < tableau[indexMin]
                indexMin ← j
            FIN SI
        FIN POUR

        // Échanger l'élément minimum avec l'élément en position i
        SI indexMin ≠ i
            temp ← tableau[i]
            tableau[i] ← tableau[indexMin]
            tableau[indexMin] ← temp
        FIN SI
    FIN POUR
FIN FONCTION
```

Explication :

- **Boucle externe (i)** : position où placer le prochain minimum
- **Boucle interne (j)** : cherche le minimum dans le reste du tableau
- **indexMin** : mémorise la position du minimum trouvé
- Un seul échange par passage

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Visualisation détaillée

Trions le tableau : `[64, 25, 12, 22, 11]`

Passage 1 : Trouver le minimum et le placer en position 0

[64, 25, 12, 22, 11]

i=0

Chercher le minimum de index 0 à 4 :

indexMin commence à 0 (valeur 64)

j=1 : 25 < 64 → indexMin = 1

j=2 : 12 < 25 → indexMin = 2

j=3 : 22 > 12 → indexMin reste 2

j=4 : 11 < 12 → indexMin = 4

Minimum = 11 (index 4)

Échanger tableau[0] et tableau[4]

[11, 25, 12, 22, 64]

Passage 2 : Trouver le minimum de position 1 à 4

[11, 25, 12, 22, 64]

i=1

Chercher le minimum de index 1 à 4 :

indexMin commence à 1 (valeur 25)

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

```
j=2 : 12 < 25 → indexMin = 2
j=3 : 22 > 12 → indexMin reste 2
j=4 : 64 > 12 → indexMin reste 2

Minimum = 12 (index 2)
Échanger tableau[1] et tableau[2]

[11, 12, 25, 22, 64]
```

Passage 3 : Trouver le minimum de position 2 à 4

```
[11, 12, 25, 22, 64]
i=2

Chercher le minimum de index 2 à 4 :
indexMin commence à 2 (valeur 25)
j=3 : 22 < 25 → indexMin = 3
j=4 : 64 > 22 → indexMin reste 3

Minimum = 22 (index 3)
Échanger tableau[2] et tableau[3]

[11, 12, 22, 25, 64]
```

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☑ Jérôme CHRETIENNE
☑ Sophie POULAKOS
☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Passage 4 : Trouver le minimum de position 3 à 4

[11, 12, 22, 25, 64]

i=3

Chercher le minimum de index 3 à 4 :

indexMin commence à 3 (valeur 25)

j=4 : 64 > 25 → indexMin reste 3

Minimum = 25 (déjà en place)

Pas d'échange

[11, 12, 22, 25, 64]

Tableau trié !

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

☒ Jérôme CHRETIENNE
☒ Sophie POULAKOS
☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

4- Comparaison des deux méthodes

Nombre de comparaisons

Pour un tableau de **N éléments** :

Algorithme	Comparaisons	Échanges (pire cas)
Tri à bulles	$N \times (N-1) / 2$	$N \times (N-1) / 2$
Tri par sélection	$N \times (N-1) / 2$	$N - 1$

Observation :

- Même nombre de comparaisons
- Tri par sélection fait **moins d'échanges**

Lisibilité

Tri à bulles :

- Concept simple (comparer voisins)
- Facile à expliquer
- Beaucoup d'échanges

Tri par sélection :

- Peu d'échanges
- Logique claire (trouver le min)
- Légèrement plus complexe à implémenter

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☑ Jérôme CHRETIENNE
- ☑ Sophie POULAKOS
- ☑ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

Introduction intuitive à la complexité

Complexité = combien de temps prend l'algorithme selon la taille des données

Notation informelle :

- **$O(n)$** : temps proportionnel à la taille (linéaire)
- **$O(n^2)$** : temps proportionnel au carré de la taille (quadratique)

Tri à bulles et tri par sélection : tous deux en **$O(n^2)$**

- Pour 10 éléments : ~100 opérations
- Pour 100 éléments : ~10 000 opérations
- Pour 1000 éléments : ~1 000 000 opérations

Ils deviennent **lents** sur de grandes quantités de données.

Note : Il existe des algorithmes plus rapides (QuickSort, MergeSort) en $O(n \log n)$, mais ils sont plus complexes.

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.

Algorithmie – Algorithmes de Tri

5- Exercices pratiques

Exercice 1 : Tri à bulles sur petit tableau

Soit le tableau : `[5, 1, 4, 2, 8]`

1. Appliquer l'algorithme du tri à bulles manuellement
2. Tracer chaque passage complet
3. Indiquer tous les échanges effectués

Exercice 2 : Tri par sélection guidé

Soit le tableau : `[29, 10, 14, 37, 13, 25]`

1. Appliquer l'algorithme du tri par sélection
2. À chaque passage, identifier le minimum trouvé
3. Indiquer l'échange effectué
4. Tracer l'état du tableau après chaque passage

Exercice 3 : Trier puis chercher

Soit le tableau : `[45, 12, 89, 23, 67, 34, 5, 78]`

1. Trier ce tableau en utilisant le tri à bulles OU le tri par sélection (au choix)
2. Une fois trié, utiliser la recherche dichotomique pour trouver le nombre **67**
3. Tracer l'exécution de la recherche dichotomique

Notions travaillées :

- Synthèse tri + recherche
- Enchaînement d'algorithmes
- Importance du tri pour la recherche dichotomique

Auteur :

Yoann DEPRIESTER

Relu, validé & visé par :

- ☒ Jérôme CHRETIENNE
- ☒ Sophie POULAKOS
- ☒ Mathieu PARIS

Date création :

xx-xx-20xx

Date révision :

xx-xx-20xx



Toute reproduction, représentation, diffusion ou rediffusion, totale ou partielle, de ce document ou de son contenu par quelque procédé que ce soit est interdite sans l'autorisation expresse, écrite et préalable de l'ADRAR.